

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO2: *Neural Network Representation, Perceptron Model, Multilayer Perceptrons, Backpropagation Algorithm, Deep Neural Networks, Genetic Algorithms, Hypothesis Space Search, Genetic Programming, Models of Evaluation and Learning (BL1, BL2)*

Assessment Tool Weightage: 20%

QUESTIONS: 15

Total Marks:25

Annexure A

SCENARIO-BASED QUESTIONS

Code of Conduct:

*I A.G.MANOJ/192312179 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: A.G.MANOJ

Annexure A

Short Answer Test

1. With the help of a neat sketch, explain the different functional blocks present in a machine learning system, including data collection, preprocessing, feature extraction, model training, and evaluation. Illustrate the working of the system by providing suitable examples for both supervised and unsupervised learning approaches, clearly highlighting how labelled and unlabelled data are handled within the learning process. (5 Marks)

2. Design a machine learning system for disaster management to predict events such as floods and earthquakes using both historical and real-time data sources. Explain the selection of an appropriate model, including whether a neural network or any other algorithm is suitable for this task, along with justification. Describe the data preprocessing steps and feature selection techniques required to improve model performance. Further, explain the complete process of training, validating, and evaluating the model, and discuss how such a system can assist authorities in early warning, risk assessment, and effective emergency response. (5 Marks)

3. In the context of neural networks, consider a multilayer perceptron used for binary classification. Explain how the backpropagation algorithm updates the weights in the network during training. Illustrate the process using a simple example by describing the forward pass, error calculation, and backward propagation of errors. Discuss how learning rate and activation functions influence the convergence and accuracy of the model. (5 Marks)

4. In the context of Genetic Algorithms, consider a dataset representing different candidate solutions for maximizing a fitness function based on two parameters: Speed and Accuracy. Each solution is evaluated as either suitable (Yes) or not suitable (No). Using the Genetic Algorithm approach, analyze how the population evolves by applying selection, crossover, and mutation operations. Show the step-by-step improvement of candidate solutions across generations and identify the best optimal solution. (5 Marks)

Dataset:

Example	Speed	Accuracy	Fitness Value	Suitable Solution
1	High	Low	40	No
2	Medium	Medium	60	Yes
3	High	Medium	75	Yes
4	Low	High	65	Yes
5	Medium	High	80	Yes

Explain how selection chooses the best individuals, how crossover generates new offspring, and how mutation helps in exploring the hypothesis space. Also, provide the final optimized solution after evolution.

5. In the context of Neural Networks for classification, consider a dataset used to predict whether a patient has a disease based on attributes such as Age, Blood Pressure, Glucose Level, and Lifestyle Risk. The output indicates whether the disease is present (Yes) or not (No). Using this dataset, explain how a neural network model can be trained using backpropagation to classify the data. Also, evaluate the model performance by comparing actual and predicted outputs. (5 Marks)

Dataset:

Example	Age	Blood Pressure	Glucose Level	Lifestyle Risk	Actual Output	Predicted Output
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Using the above dataset, compute the values of True Positives, True Negatives, False Positives, and False Negatives. Based on these values, analyze the performance of the neural network using suitable evaluation metrics. Also, explain how preprocessing and feature selection improve the model's accuracy and reliability.

(5 Marks)

6. Design a machine learning system for smart traffic management to predict traffic congestion using data such as vehicle density, time of day, weather conditions, and road type. Justify the choice of algorithm, explain preprocessing and feature engineering steps, and describe how the model can be trained, validated, and deployed for real-time decision-making. (5 Marks)

7. In the context of **Convolutional Neural Networks (CNNs)**, explain the different layers involved, including convolutional, pooling, and fully connected layers. Describe how feature maps are generated and how backpropagation updates weights in CNNs. Illustrate the process with an example of image classification and discuss factors affecting model performance. (5 Marks)

8. Consider the application of **Genetic Algorithms for feature selection** in a machine learning problem. Explain how candidate feature subsets are represented as chromosomes and how fitness is evaluated. Describe the role of selection, crossover, and mutation in evolving better feature subsets, and discuss how this improves model accuracy and reduces complexity. (5 Marks)

9. In the context of **classification problems**, explain how different evaluation metrics such as accuracy, precision, recall, F1-score, and ROC curve are used to assess model performance. Using a sample confusion matrix, demonstrate the calculation of these metrics and discuss their importance in applications such as medical diagnosis and fraud detection. (5 Marks)

10. Consider the use of **Genetic Algorithms for optimization problems** such as scheduling tasks in a system. Explain how solutions are encoded as chromosomes

and how the fitness function is defined. Describe the processes of selection, crossover, and mutation, and discuss how these operations help in finding near-optimal solutions efficiently. (5 Marks)

SOLUTIONS

Q1. Functional Blocks of a Machine Learning System

Functional Pipeline (Sketch)

Data Collection	Preprocessing	Feature Extraction	Model Training	Evaluation
-----------------	---------------	--------------------	----------------	------------

Each block feeds into the next, and the trained model ultimately supports Deployment / Prediction on new data.

Explanation of Blocks:

- Data Collection – Gathering raw data from sensors, databases, APIs, logs, or manual surveys. Quality and volume directly affect downstream performance.
- Preprocessing – Cleaning (removing duplicates/noise), handling missing values, normalizing/standardizing numeric ranges, and encoding categorical variables.
- Feature Extraction/Selection – Transforming raw attributes into informative features (e.g., PCA, statistical aggregates) and removing irrelevant/redundant ones.
- Model Training – A learning algorithm (decision tree, neural network, SVM, etc.) is fit to training data by optimizing a loss function.
- Evaluation – The trained model is tested on unseen data using metrics such as accuracy, precision/recall, or RMSE to judge generalization.

Supervised vs Unsupervised Example:

Supervised (labelled data): Predicting house prices from features (area, location) with known price labels. The model learns a mapping $f(X) \rightarrow Y$ by minimizing error between predicted and actual (labelled) prices — e.g., linear regression or a neural network trained with backpropagation.

Unsupervised (unlabelled data): Customer segmentation from purchase data with no predefined labels. A clustering algorithm (e.g., K-Means) groups similar customers based on feature similarity alone, discovering structure without ground-truth targets.

Key distinction: supervised learning uses the label to compute an error signal for training; unsupervised learning relies purely on the inherent structure/distribution of the input data.

Q2. ML System for Disaster Management (Flood/Earthquake Prediction)

1. Data Sources:

Historical: past flood/earthquake records, rainfall/seismic archives, terrain/elevation maps. Real-time: IoT sensors (water level, seismographs), satellite imagery, weather feeds (rainfall, humidity, wind), social media reports.

2. Model Selection & Justification:

A hybrid approach is recommended — an LSTM/RNN (or deep neural network) for time-series sensor data (rainfall trends, seismic tremors), combined with a Random Forest / Gradient Boosting model for tabular, static features (soil type, elevation, historical frequency).

Justification: neural networks (especially LSTMs) capture temporal dependencies in continuously streaming sensor data — essential for early warning — while tree-based ensembles handle heterogeneous tabular features robustly and need less labelled data, useful since disaster events are relatively rare.

3. Preprocessing:

Handling missing sensor readings (interpolation), normalizing scales (mm rainfall vs. Richter magnitude), removing outlier sensor noise, and time-aligning multi-source streams.

4. Feature Selection:

Correlation analysis and domain knowledge select predictive features: rainfall intensity over past N hours, river water level trend, soil saturation, tectonic plate movement rate, historical seismic frequency. Techniques such as Recursive Feature Elimination (RFE) or tree-based feature-importance trim redundant sensors.

5. Training, Validation, Evaluation:

- Split data chronologically (train on older events, validate/test on recent ones) to avoid data leakage.
- Use k-fold or time-series cross-validation to tune hyperparameters.
- Evaluate with recall-focused metrics (missing a real disaster is far costlier than a false alarm) — Recall, F1-score, Precision-Recall AUC rather than plain accuracy.

6. Operational Use:

The model outputs a risk score/probability, thresholded to trigger tiered alerts — assisting authorities in early warning (advance evacuation), risk assessment (mapping high-risk zones for resource pre-positioning), and emergency response coordination (real-time dashboards showing affected areas ranked by predicted severity).

Q3. Backpropagation in a Multilayer Perceptron (Binary Classification)

Network: Input layer → Hidden layer (with activation, e.g., sigmoid/ReLU) → Output neuron (sigmoid, for binary output 0/1).

1. Forward Pass:

Input x is multiplied by weights $W1$ and passed through the hidden-layer activation: $h = \sigma(W1 \cdot x + b1)$. The hidden output h is multiplied by $W2$ and passed through the output activation: $\hat{y} = \sigma(W2 \cdot h + b2)$.

2. Error Calculation:

Loss is computed, typically binary cross-entropy: $L = -[y \cdot \log(\hat{y}) + (1-y) \cdot \log(1-\hat{y})]$, where y is the true label.

3. Backward Propagation:

- Gradient of loss w.r.t. output: $\delta_{\text{output}} = (\hat{y} - y)$.
- Propagate error to hidden layer: $\delta_{\text{hidden}} = (\delta_{\text{output}} \cdot W2) \odot \sigma'(h)$ (elementwise, using the activation derivative).
- Weight gradients: $\partial L / \partial W2 = \delta_{\text{output}} \cdot h^T$, $\partial L / \partial W1 = \delta_{\text{hidden}} \cdot x^T$.

- Update weights: $W = W - \eta \cdot \partial L / \partial W$, where η is the learning rate.

Worked Illustration:

Suppose $x = 1$, target $y = 1$, and the network initially outputs $\hat{y} = 0.3$. The error $(\hat{y} - y) = -0.7$ is large, so weight updates push future predictions closer to 1, scaled by the learning rate and the local gradient of the sigmoid at that point.

Effect of Learning Rate and Activation Function:

Learning rate (η): Too large causes overshooting/oscillation or divergence; too small causes very slow convergence and risk of getting stuck. A well-tuned rate (often with decay schedules) balances speed and stability.

Activation function: Sigmoid/tanh can cause vanishing gradients in deep networks (gradients shrink as they propagate backward), slowing learning. ReLU mitigates this for hidden layers, generally improving convergence speed, though it can suffer from "dead neurons." The choice directly affects both convergence speed and final accuracy.

Q4. Genetic Algorithm — Speed/Accuracy Optimization

Dataset:

Ex	Speed	Accuracy	Fitness	Suitable
1	High	Low	40	No
2	Medium	Medium	60	Yes
3	High	Medium	75	Yes
4	Low	High	65	Yes
5	Medium	High	80	Yes

Step 1 — Encoding (Chromosome Representation):

Each solution is represented as a 4-bit chromosome: 2 bits for Speed (Low=00, Medium=01, High=10) + 2 bits for Accuracy (Low=00, Medium=01, High=10).

Ex	Chromosome	Fitness
2	01 01	60
3	10 01	75
4	00 10	65
5	01 10	80

(Example 1 is discarded early — marked "No" with the lowest fitness.)

Step 2 — Selection:

Using fitness-proportionate (roulette-wheel) selection, individuals with higher fitness are more likely chosen as parents. Example 5 (fitness 80) and Example 3 (fitness 75) are the fittest and are selected as parents:

- Parent A (Ex 5): 01 | 10 → Medium Speed, High Accuracy
- Parent B (Ex 3): 10 | 01 → High Speed, Medium Accuracy

Step 3 — Crossover:

Applying single-point crossover at the boundary between the Speed gene and the Accuracy gene:

- Offspring 1 = Speed(A) + Accuracy(B) = 01 01 → Medium Speed, Medium Accuracy
- Offspring 2 = Speed(B) + Accuracy(A) = 10 10 → High Speed, High Accuracy

Offspring 2 combines the best trait from each parent (High Speed from B, High Accuracy from A), and is expected to have the highest fitness of the generation — potentially exceeding both parents

(illustratively, fitness $\approx 90+$).

Step 4 — Mutation:

A small random bit-flip is applied with low probability to maintain diversity, e.g., flipping one bit of Offspring 1's Accuracy gene explores a new region of the search space that crossover alone might not reach, preventing the population from getting stuck at a local optimum.

Step 5 — Evolution over Generations

Repeating selection → crossover → mutation across generations, the average and best fitness of the population steadily increases, since weaker chromosomes (like Example 1) are progressively eliminated and stronger combinations propagate.

Final Optimized Solution: High Speed + High Accuracy (Offspring 2) — the combination that maximizes the fitness function while remaining "Suitable."

Q5. Neural Network Classification — Disease Prediction & Evaluation

Dataset:

Ex	Age	BP	Glucose	Lifestyle	Actual	Predicted
1	45	High	High	High	Yes	Yes
2	50	Normal	High	Medium	Yes	No
3	30	Normal	Normal	Low	No	No
4	60	High	High	High	Yes	Yes
5	40	Normal	Normal	Medium	No	Yes

Training Process:

Input features (Age, BP, Glucose, Lifestyle Risk — categorical ones one-hot/ordinal encoded) are fed forward through the network to produce a predicted probability of disease. Backpropagation compares this to the actual label (encoded as 1/0), computes the cross-entropy loss, and updates weights via gradient descent, iterating over epochs until the loss converges.

Confusion Matrix Computation:

Example	Actual	Predicted	Outcome
1	Yes	Yes	TP
2	Yes	No	FN
3	No	No	TN
4	Yes	Yes	TP
5	No	Yes	FP

- True Positives (TP) = 2
- True Negatives (TN) = 1
- False Positives (FP) = 1
- False Negatives (FN) = 1

Evaluation Metrics:

- Accuracy = $(TP+TN)/\text{Total} = (2+1)/5 = 0.60$ (60%)
- Precision = $TP/(TP+FP) = 2/3 = 0.667$ (66.7%)
- Recall (Sensitivity) = $TP/(TP+FN) = 2/3 = 0.667$ (66.7%)
- F1-score = $2 \cdot (\text{Precision} \cdot \text{Recall}) / (\text{Precision} + \text{Recall}) = 0.667$

Interpretation:

The model correctly identifies 60% of cases overall; precision and recall are equal (0.667), indicating a moderate but not ideal balance — it misses one true disease case (FN) and raises one false alarm (FP). In a medical context, recall is especially critical, so this model would benefit from further tuning to reduce false negatives.

Role of Preprocessing & Feature Selection:

Normalizing continuous features (Age, Glucose) prevents any single feature from dominating due to scale; encoding categorical values (BP, Lifestyle Risk) consistently allows the network to learn meaningful weight patterns; removing irrelevant/noisy features reduces overfitting and improves generalization, directly raising both accuracy and reliability on unseen patients.

Q6. ML System for Smart Traffic Management

1. Data Sources:

Vehicle density (loop detectors/cameras), time of day, weather conditions (rain, fog), road type (highway/arterial/residential), historical congestion patterns.

2. Algorithm Choice & Justification:

A Gradient Boosted Trees model (e.g., XGBoost) or an LSTM-based neural network is suitable — LSTM captures the sequential/time-dependent nature of traffic flow (rush-hour patterns), while boosted trees handle mixed categorical + numeric features well and train faster on moderate data volumes. For real-time deployment, a lighter model is often preferred over deep LSTMs due to latency constraints.

3. Preprocessing:

Handling missing sensor data (interpolation), normalizing vehicle counts, encoding categorical weather/road-type features, aggregating data into time windows (e.g., 5-minute intervals).

4. Feature Engineering:

Creating derived features such as "average speed in last 15 min," "day-of-week," "is-holiday" flag, and interaction terms (e.g., weather × time-of-day) to capture compound effects on congestion.

5. Training/Validation:

Time-based train/validation/test split (train on past months, validate on recent weeks) reflecting real deployment conditions; hyperparameter tuning via cross-validation; evaluate using RMSE/MAE (for congestion-level regression) or accuracy/F1 (if framed as congestion/no-congestion classification).

6. Deployment:

The trained model is deployed as a lightweight inference service integrated with traffic signal controllers, providing real-time congestion predictions that dynamically adjust signal timing, reroute suggestions, and public alerts — with periodic retraining as new data arrives to handle concept drift (e.g., seasonal changes).

Q7. Convolutional Neural Networks (CNNs)

Layers:

Convolutional Layer: Applies learnable filters (kernels) that slide over the input image, computing dot products to produce feature maps highlighting patterns like edges, textures, or shapes. Each filter learns to detect a different feature.

Pooling Layer (e.g., Max Pooling): Downsamples feature maps (e.g., taking the max value in each 2×2 region), reducing spatial dimensions and computational cost while retaining salient features and providing some translation invariance.

Fully Connected (Dense) Layer: Flattens the final feature maps into a vector and connects every neuron to the next layer, combining extracted features to make the final classification decision (e.g., via a softmax output).

Feature Map Generation:

Each convolutional filter convolves across the image, and the resulting activation (after applying a non-linearity like ReLU) forms one feature map; stacking many filters per layer produces a stack of feature maps capturing diverse patterns, which become progressively more abstract (edges → textures → object parts) in deeper layers.

Backpropagation in CNNs:

Similar to MLPs, the error at the output is propagated backward through the fully connected layers, then through pooling layers (routing gradients only through the max-selected positions in max pooling), and finally through convolutional layers, where gradients update each filter's weights via the chain rule, adjusted for the shared-weight (convolution) structure.

Example — Image Classification (Cat vs Dog):

The input image passes through several conv+pool blocks extracting increasingly abstract features (fur texture → ear shape → overall silhouette), then a fully connected layer combines these into a final "cat" or "dog" probability. Errors between predicted and true labels are backpropagated to refine filters over training epochs.

Factors Affecting Performance:

Depth/architecture of the network, quality and quantity of training data (including augmentation), choice of activation functions, learning rate, regularization (dropout, batch normalization), and computational resources for training larger models.

Q8. Genetic Algorithms for Feature Selection

Chromosome Representation:

Each candidate feature subset is encoded as a binary string, where each bit position corresponds to a feature (1 = included, 0 = excluded). For 5 features [Age, BP, Glucose, Lifestyle, Gender], a chromosome 10110 selects Age, Glucose, and Lifestyle.

Fitness Evaluation:

The fitness of each chromosome is typically the classification accuracy (or F1-score) obtained by training a model using only the selected features, optionally penalized for including too many features to encourage compact, efficient subsets.

Evolutionary Operations:

Selection: Chromosomes (feature subsets) with higher fitness (better resulting model accuracy) are more likely selected as parents, e.g., via roulette-wheel or tournament selection.

Crossover: Two parent bit-strings exchange segments (single-point or uniform crossover) to produce offspring subsets that combine promising features from both parents.

Mutation: Randomly flips a bit (toggling a feature in/out), introducing new feature combinations and preventing premature convergence to a suboptimal subset.

Benefit:

Over generations, subsets with irrelevant/redundant features are weeded out (low fitness), while informative feature combinations propagate — improving model accuracy (by removing noisy features) and reducing complexity/training time (by shrinking the feature space), which also lowers the risk of overfitting.

Q9. Evaluation Metrics for Classification:

Sample Confusion Matrix:

	Predicted Positive	Predicted Negative
Actual Positive	TP = 40	FN = 10
Actual Negative	FP = 5	TN = 45

Calculations:

- Accuracy = $(TP+TN)/Total = (40+45)/100 = 0.85$
- Precision = $TP/(TP+FP) = 40/45 = 0.889$
- Recall = $TP/(TP+FN) = 40/50 = 0.80$
- F1-score = $2 \cdot (Precision \cdot Recall) / (Precision + Recall) \approx 0.842$
- ROC Curve: plots True Positive Rate (Recall) against False Positive Rate ($FP/(FP+TN)$) at varying thresholds; AUC (Area Under Curve) summarizes discriminative ability — closer to 1 is stronger, 0.5 is random guessing.

Importance in Applications:

Medical diagnosis: Recall is prioritized — missing a disease case (false negative) can be life-threatening, so models are tuned to minimize FN even at some cost to precision.

Fraud detection: Precision often matters more, since too many false positives erode customer trust, though recall still matters to catch actual fraud; F1-score balances both, and ROC-AUC helps compare models independent of a fixed threshold.

Q10. Genetic Algorithms for Task Scheduling Optimization

Encoding:

Each chromosome represents a candidate schedule — e.g., a permutation of task IDs indicating the order (and possibly the machine/resource) in which tasks are executed, such as [T3, T1, T4, T2].

Fitness Function:

Defined based on the scheduling objective — e.g., minimizing total completion time (makespan), minimizing resource idle time, or maximizing throughput. Lower makespan (or higher throughput) yields higher fitness.

Evolutionary Process:

Selection: Schedules with lower makespan (higher fitness) are more likely chosen as parents via tournament or roulette-wheel selection.

Crossover: Since chromosomes are permutations, order-preserving crossover techniques (e.g., Order Crossover — OX) combine sub-sequences from two parent schedules to produce valid offspring schedules that inherit good task orderings from both parents.

Mutation: Randomly swaps the position of two tasks in the schedule, introducing variation and helping escape local optima.

Outcome:

Across generations, schedules with poor fitness (long makespans, resource conflicts) are eliminated, while efficient orderings propagate and recombine. This iterative evolutionary search efficiently explores a huge combinatorial space of possible schedules and converges toward a near-optimal task schedule — much faster than exhaustive search for large problem sizes.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand